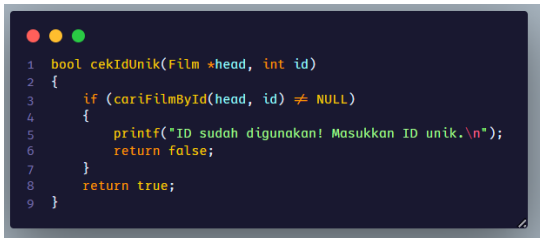
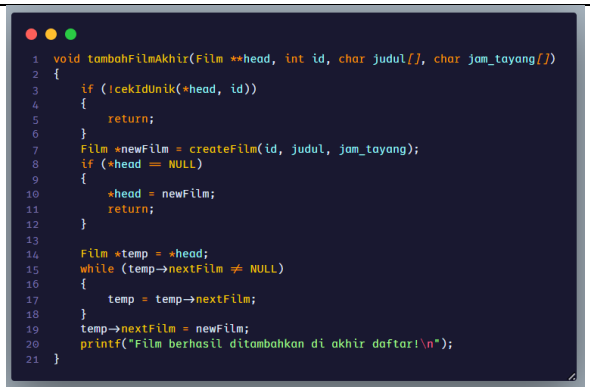
IK141 Struktur Data
Singly Linked List

Nama : Novan Arrijal Ghifari Hakim S
NIM : 2508836
Kelas : Pilkom B

PROGRAM STUDI PENDIDIKAN ILMU KOMPUTER
FAKULTAS PENDIDIKAN MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS PENDIDIKAN INDONESIA
2026

A. Potongan Kode

No	Source Code
Soal 1	
1	 Pada soal no 1 saya menggunakan 1 single link list bernama film yang anggotanya adalah id judul, jam tayang dan alamat ke film berikutnya.
2	 Untuk menambahkan data di awal, saya menggunakan fungsi tambahFilmAwal, di dalamnya akan menjalankan pengencekan id, kemudian menambahkan data film di awal.  Isi dari fungsi pengecekan id memanggil fungsi cariFilmById yang didalam nya mengecek apakah ada id yang cari pada list.
3	 Kemudian untuk melakukan tambah akhir, saya melakukan pengecekan id, kemudian membuat variabel newfilm dan akan disimpan di akhir dari list jika head nya tidak kosong.

```c
typedef struct Film
{
    int id;
    char judul[100];
    char jam_tayang[10];
    struct Film *nextFilm;
} Film;
```

```c
void tambahFilmAwal(Film **head, int id, char judul[], char jam_tayang[])
{
    if (!cekIdUnik(*head, id))
    {
        return;
    }

    Film *newFilm = createFilm(id, judul, jam_tayang);
    newFilm->nextFilm = *head;
    *head = newFilm;
    printf("Film berhasil ditambahkan di awal daftar!\n");
}
```

```c
bool cekIdUnik(Film *head, int id)
{
    if (cariFilmById(head, id) != NULL)
    {
        printf("ID sudah digunakan! Masukkan ID unik.\n");
        return false;
    }
    return true;
}
```

```c
Film *cariFilmById(Film *head, int id)
{
    Film *temp = head;
    while (temp != NULL)
    {
        if (temp->id == id)
        {
            return temp;
        }
        temp = temp->nextFilm;
    }
    return NULL;
}
```

```c
void tambahFilmAkhir(Film **head, int id, char judul[], char jam_tayang[])
{
    if (!cekIdUnik(*head, id))
    {
        return;
    }
    Film *newFilm = createFilm(id, judul, jam_tayang);
    if (*head == NULL)
    {
        *head = newFilm;
        return;
    }

    Film *temp = *head;
    while (temp->nextFilm != NULL)
    {
        temp = temp->nextFilm;
    }
    temp->nextFilm = newFilm;
    printf("Film berhasil ditambahkan di akhir daftar!\n");
}
```

4

```c
void sisipkanFilmSetelah(Film **head, int id, char judul[], char jam_tayang[], int id_target)
{
    if (!cekIdUnik(*head, id))
    {
        return;
    }
    Film *newFilm = createFilm(id, judul, jam_tayang);
    Film *temp = cariFilmById(*head, id_target);
    if (temp == NULL)
    {
        printf("ID target tidak ditemukan!\n");
        free(newFilm);
        return;
    }
    newFilm->nextFilm = temp->nextFilm;
    temp->nextFilm = newFilm;
    printf("Film berhasil disisipkan setelah ID %d!\n", id_target);
}
```

Fitur selanjutnya adalah sisipkan setelah/sebelum, nah fungsi diatas adlah fungsi sisipkan setelah, cara kerja nya fungsi akan mengecek id unik terlebih dahulu, dan mencari film yang ditargetkan dan menambahkan film baru setelahnya.

5

```c
void sisipkanFilmSebelum(Film **head, int id, char judul[], char jam_tayang[], int id_target)
{
    if (!cekIdUnik(*head, id))
    {
        return;
    }
    Film *newFilm = createFilm(id, judul, jam_tayang);
    if (*head == NULL || (*head)->id == id_target)
    {
        newFilm->nextFilm = *head;
        *head = newFilm;
        return;
    }

    Film *temp = *head;
    while (temp->nextFilm != NULL && temp->nextFilm->id != id_target)
    {
        temp = temp->nextFilm;
    }
    if (temp->nextFilm == NULL)
    {
        printf("ID target tidak ditemukan!\n");
        free(newFilm);
        return;
    }

    newFilm->nextFilm = temp->nextFilm;
    temp->nextFilm = newFilm;
    printf("Film berhasil disisipkan sebelum ID %d!\n", id_target);
}
```

Untuk sisipkan film sebelum, cara kerja nya mirip dengan sisip setelah, melakukan pengecekan id, kemudian mencari target. Bedanya disini dicari dengan temp->nextFilm->id agar tidak memerlukan variabel lagi seperti prev, dengan begitu eksekusi memasukkan film baru bisa hanya dengan 2 baris kode.

6

```c
void updateDataFilm(Film *head, int id, char judul_baru[], char jam_tayang_baru[])
{
    Film *temp = cariFilmById(head, id);
    if (temp == NULL)
    {
        printf("ID tidak ditemukan!\n");
        return;
    }
    strcpy(temp->judul, judul_baru);
    strcpy(temp->jam_tayang, jam_tayang_baru);
    printf("Data film dengan ID %d berhasil diperbarui!\n", id);
}
```

Kemudian untuk fitur update film menggunakan fungsi diatas, fungsi itu akan mencari film menggunakan cariFilmById kemudian merubah data sesuai inputnya.

7

```c
void hapusJadwalAwal(Film **head)
{
    if (*head == NULL)
    {
        printf("Jadwal tayang kosong!\n");
        return;
    }
    Film *temp = *head;
    *head = (*head)->nextFilm;
    free(temp);
    printf("Jadwal pertama berhasil dihapus!\n");
}
```

Kemdudian untuk fitur hapus awal fungsi akan langsung menghapus head dan mengubah head menjadi head->nextFilm.

```c
void hapusJadwalAkhir(Film **head)
{
    if (*head == NULL)
    {
        printf("Jadwal tayang kosong!\n");
        return;
    }
    if ((*head)->nextFilm == NULL)
    {
        free(*head);
        *head = NULL;
        return;
    }
    Film *temp = *head;
    while (temp->nextFilm->nextFilm != NULL)
    {
        temp = temp->nextFilm;
    }
    free(temp->nextFilm);
    temp->nextFilm = NULL;
    printf("Jadwal terakhir berhasil dihapus!\n");
}
```

Untuk hapus akhir, fungsi akan mencari hingga temp->nextfilm->nextfilm nya null, itu dilakukan agar tidak menggunakan variabel prev sehingga ketika sudah null tinggal menghapus temp->nextfilm dan megubah temp->nextFilm menjadi NULL.

8

```c
Film *head = NULL;
tambahFilmAkhir(&head, 101, "Avatar Endgame", "14:00");
tambahFilmAkhir(&head, 102, "KKN di Desa Penari", "16:00");
int pilihan;
int id;
char judul[100];
char jam_tayang[10];
```

```c
printf("=== Manajemen Bioskop=== \n");
printf("1. Tambah Film di Awal\n");
printf("2. Tambah Film di Akhir\n");
printf("3. Sisipkan Film (Sebelum/Sesudah ID tertentu)\n");
printf("4. Lihat Jadwal Tayang\n");
printf("5. Perbarui Data Film (berdasarkan ID)\n");
printf("6. Hapus Jadwal (Awal/Akhir)\n");
printf("7. Keluar\n");
printf("Pilih menu: ");
scanf("%d", &pilihan);
```

Pada bagian main.c terdapat deklarasi variabel dan isi dari menu nya. Kemudian ada switch untuk eksekusi pilihan dari user, isi switch nya bisa dilihat dibawah ini :

```c
switch (pilihan)
{
case 1:
    templateInput(&id, judul, jam_tayang);
    tambahFilmAwal(&head, id, judul, jam_tayang);
    break;
case 2:
    templateInput(&id, judul, jam_tayang);
    tambahFilmAkhir(&head, id, judul, jam_tayang);
    break;
case 3:
    printf("Pilih opsi sisipkan: \n1. Setelah ID tertentu \n2. Sebelum ID tertentu\n");
    int opsi_sisip;
    printf("Pilihan: ");
    scanf("%d", &opsi_sisip);
    printf("Masukkan ID target: ");
    int id_target;
    scanf("%d", &id_target);
    templateInput(&id, judul, jam_tayang);
    if (opsi_sisip == 1)
    {
        sisipkanFilmSetelah(&head, id, judul, jam_tayang, id_target);
    }
    else if (opsi_sisip == 2)
    {
        sisipkanFilmSebelum(&head, id, judul, jam_tayang, id_target);
    }
    else
    {
        printf("Opsi tidak valid!\n");
    }
    break;
```

```c
case 4:
    lihatJadwalTayang(head);
    break;
case 5:
    printf("Masukkan ID film yang ingin diperbarui: ");
    int id_update;
    scanf("%d", &id_update);
    printf("Masukkan Judul Film: ");
    scanf(" %[^\n]", judul);
    printf("Masukkan Jam Tayang (HH:MM): ");
    scanf(" %[^\n]", jam_tayang);
    updateDataFilm(head, id_update, judul, jam_tayang);
    break;
case 6:
    printf("Pilih opsi hapus: \n1. Awal \n2. Akhir\n");
    printf("Pilihan: ");
    int opsi_hapus;
    scanf("%d", &opsi_hapus);
    if (opsi_hapus == 1)
    {
        hapusJadwalAwal(&head);
    }
    else if (opsi_hapus == 2)
    {
        hapusJadwalAkhir(&head);
    }
    else
    {
        printf("Opsi tidak valid!\n");
    }
    break;
default:
    printf("Pilihan tidak valid!\n");
    break;
}
```

Soal 2

1

```c
typedef struct Pelanggan
{
    int no_antrean;
    char nama[30];
    char pesanan[100];
    struct Pelanggan *nextPelanggan;
} Pelanggan;
```

ADT yang digunakan di soal ini hanya 1 yaitu Pelanggan dengan anggota no_antrean, nama, pesanan, dan alamat pelanggan selanjutnya.

2

```c
void templateInput(int *no_antrean, char nama[], char pesanan[])
{
    printf("Masukkan Nomor Pelanggan: ");
    scanf("%d", no_antrean);
    printf("Masukkan Nama Pelanggan: ");
    scanf(" %[^\n]", nama);
    printf("Masukkan Pesanan: ");
    scanf(" %[^\n]", pesanan);
}
```

Untuk melakukan input saya membuat fungsi templateInput agar tidak mengulang kode yang sama terus menerus. Begitupun dengan createpelanggan

```c
Pelanggan *createPelanggan(int no_antrean, char nama[], char pesanan[])
{
    Pelanggan *newPelanggan = (Pelanggan *)malloc(sizeof(Pelanggan));
    newPelanggan->no_antrean = no_antrean;
    strcpy(newPelanggan->nama, nama);
    strcpy(newPelanggan->pesanan, pesanan);
    newPelanggan->nextPelanggan = NULL;
    return newPelanggan;
}
```

3

```c
Pelanggan *cariPelangganByNomor(Pelanggan *head, int no_antrean)
{
    Pelanggan *temp = head;
    while (temp != NULL)
    {
        if (temp->no_antrean == no_antrean)
        {
            return temp;
        }
        temp = temp->nextPelanggan;
    }
    return NULL;
}
```

Lalu untuk mencari pelanggan saya membuat fungsi terpisah yang mengembalikan data pelanggan jika ada.

4

```c
bool cekNomorUnik(Pelanggan *head, int no_antrean)
{
    if (cariPelangganByNomor(head, no_antrean) != NULL)
    {
        printf("Nomor sudah digunakan!\n");
        return false;
    }
    return true;
}
```

Saya juga membuat fungsi khusus pengecekan nomor antrean sehingga nomor antrean tetap unik dan tidak bsia digunakan oleh pelanggan lainnya

5

```c
void tambahPelangganAwal(Pelanggan **head, int no_antrean, char nama[], char pesanan[])
{
    if (!cekNomorUnik(*head, no_antrean))
    {
        return;
    }

    Pelanggan *newPelanggan = createPelanggan(no_antrean, nama, pesanan);
    newPelanggan->nextPelanggan = *head;
    *head = newPelanggan;
    printf("Pelanggan berhasil ditambahkan di awal daftar!\n");
}
```

Untuk melakukan tampah pelanggan VIP digunakan fungsi tambahPelangganAwal yang akan mengecek id kemudian membuat pelanggan baru dan mengubah head menjadi pelanggan baru.

6

```c
void tambahPelangganAkhir(Pelanggan **head, int no_antrean, char nama[], char pesanan[])
{
    if (!cekNomorUnik(*head, no_antrean))
    {
        return;
    }
    Pelanggan *newPelanggan = createPelanggan(no_antrean, nama, pesanan);
    if (*head == NULL)
    {
        *head = newPelanggan;
        return;
    }

    Pelanggan *temp = *head;
    while (temp->nextPelanggan != NULL)
    {
        temp = temp->nextPelanggan;
    }
    temp->nextPelanggan = newPelanggan;
    printf("Pelanggan berhasil ditambahkan di akhir daftar!\n");
}
```

Untuk menambahkan pelanggan reguler, akan menggunakan fungsi tambahPelangganAkhir yang cara kerjanya mencari hingga ujung list kemudian menambahkan pelanggan baru.

7

```c
void sisipkanPelangganSetelah(Pelanggan **head, int no_antrean, char nama[], char
pesanan[], int no_target)
{
    if (!cekNomorUnik(*head, no_antrean))
    {
        return;
    }
    Pelanggan *newPelanggan = createPelanggan(no_antrean, nama, pesanan);
    Pelanggan *temp = cariPelangganByNomor(*head, no_target);
    if (temp == NULL)
    {
        printf("Nomor target antrian tidak ditemukan!\n");
        free(newPelanggan);
        return;
    }
    newPelanggan->nextPelanggan = temp->nextPelanggan;
    temp->nextPelanggan = newPelanggan;
    printf("Pelanggan berhasil disisipkan setelah Nomor %d!\n", no_target);
}
```

Fitur selanjutnya adalah sisipkan pelanggan setelah, fungsi ini akan mengecek no antrean apakah unik atau tidak kemudian mencari target pelanggan dan menambahkan pelanggan baru setelah data target.

8

```c
void sisipkanPelangganSebelum(Pelanggan **head, int no_antrean, char nama[], char pesanan[], int no_target)
{
    if (!cekNomorUnik(*head, no_antrean))
    {
        return;
    }
    Pelanggan *newPelanggan = createPelanggan(no_antrean, nama, pesanan);
    if (*head == NULL || (*head)->no_antrean == no_target)
    {
        newPelanggan->nextPelanggan = *head;
        *head = newPelanggan;
        return;
    }

    Pelanggan *temp = *head;
    while (temp->nextPelanggan != NULL && temp->nextPelanggan->no_antrean != no_target)
    {
        temp = temp->nextPelanggan;
    }
    if (temp->nextPelanggan == NULL)
    {
        printf("Nomor target antrian tidak ditemukan!\n");
        free(newPelanggan);
        return;
    }

    newPelanggan->nextPelanggan = temp->nextPelanggan;
    temp->nextPelanggan = newPelanggan;
    printf("Pelanggan berhasil disisipkan sebelum Nomor %d!\n", no_target);
}
```

Sama seperti sebelumnya untuk sisipkan sebelum saya tidak menggunakan prev melainkan mencari dengan temp->nextpelanggan->no_antrean agar saya langsung berada di posisi prev kemudian menghubungkan alamat pelanggan baru nya.

9

```c
void updateDataPelanggan(Pelanggan *head, int no_antrean, char pesanan_baru[])
{
    Pelanggan *temp = cariPelangganByNomor(head, no_antrean);
    if (temp == NULL)
    {
        printf("Nomor antrian tidak ditemukan!\n");
        return;
    }
    strcpy(temp->pesanan, pesanan_baru);
    printf("Data pelanggan dengan Nomor %d berhasil diperbarui!\n", no_antrean);
}
```

Kemudian fitur upadate, fungsi diatas akan mencari pelanggan sesuai dengan id yang diinginkan kemudian mengeksekusi perubahan data pesanan.

10

```c
void hapusPelangganByNomor(Pelanggan **head, int no_target)
{
    if (*head == NULL)
    {
        printf("Tidak ada pelanggan dalam list!\n");
        return;
    }
    if ((*head)->no_antrean == no_target)
    {
        Pelanggan *temp = *head;
        *head = (*head)->nextPelanggan;
        free(temp);
        printf("Pelanggan dengan Nomor %d berhasil dihapus!\n", no_target);
        return;
    }

    Pelanggan *temp = *head;
    while (temp->nextPelanggan != NULL && temp->nextPelanggan->no_antrean != no_target)
    {
        temp = temp->nextPelanggan;
    }
    if (temp->nextPelanggan == NULL)
    {
        printf("Nomor antrian tidak ditemukan!\n");
        return;
    }

    Pelanggan *pelanggan_hapus = temp->nextPelanggan;
    temp->nextPelanggan = pelanggan_hapus->nextPelanggan;
    free(pelanggan_hapus);
    printf("Pelanggan dengan Nomor %d berhasil dihapus!\n", no_target);
}
```

Yang terakhir ada hapus pelanggan, cara kerja fungsi ini tidak menggunakan prev juga tetapi mencari temp->nextPelanggan yang no antrean nya seusai yang kita cari, jadi yang dihapus nantinya adalah temp->nextPelanggan.

B. Screenshot Output

No	Output Program
Soal 1	
1	

```
=== Manajemen Bioskop===
1. Tambah Film di Awal
2. Tambah Film di Akhir
3. Sisipkan Film (Sebelum/Sesudah ID tertentu)
4. Lihat Jadwal Tayang
5. Perbarui Data Film (berdasarkan ID)
6. Hapus Jadwal (Awal/Akhir)
7. Keluar
Pilih menu: 1
Masukkan ID Film: 100
Masukkan Judul Film: Avengers Endgame
Masukkan Jam Tayang (HH:MM): 10:00
Film berhasil ditambahkan di awal daftar!
```

```
Pilih menu: 2
Masukkan ID Film: 109
Masukkan Judul Film: Jumbo
Masukkan Jam Tayang (HH:MM): 21:00
Film berhasil ditambahkan di akhir daftar!
```

```
Pilih menu: 3
Pilih opsi sisipkan:
1. Setelah ID tertentu
2. Sebelum ID tertentu
Pilihan: 1
Masukkan ID target: 102
Masukkan ID Film: 108
Masukkan Judul Film: Home Alone
Masukkan Jam Tayang (HH:MM): 20:00
Film berhasil disisipkan setelah ID 102!
```

```
Pilih menu: 3
Pilih opsi sisipkan:
1. Setelah ID tertentu
2. Sebelum ID tertentu
Pilihan: 2
Masukkan ID target: 108
Masukkan ID Film: 105
Masukkan Judul Film: Spider Man: Far From Home
Masukkan Jam Tayang (HH:MM): 18:00
Film berhasil disisipkan sebelum ID 108!
```

```
Pilih menu: 4
DAFTAR JADWAL TAYANG BIOSKOP:
=============================================
ID: 100 | Avengers Endgame     | Pukul: 10:00
ID: 101 | Avatar Endgame       | Pukul: 14:00
ID: 102 | KKN di Desa Penari   | Pukul: 16:00
ID: 105 | Spider Man: Far From Home | Pukul: 18:00
ID: 108 | Home Alone           | Pukul: 20:00
ID: 109 | Jumbo                | Pukul: 21:00
=============================================
```

```
Pilih menu: 5
Masukkan ID film yang ingin diperbarui: 105
Masukkan Judul Film: Spiderman: LastGame
Masukkan Jam Tayang (HH:MM): 17:00
Data film dengan ID 105 berhasil diperbarui!
```

```
Pilih menu: 6
Pilih opsi hapus:
1. Awal
2. Akhir
Pilihan: 1
Jadwal pertama berhasil dihapus!
```

```
Pilih menu: 6
Pilih opsi hapus:
1. Awal
2. Akhir
Pilihan: 2
Jadwal terakhir berhasil dihapus!
```

```
DAFTAR JADWAL TAYANG BIOSKOP:
=============================================
ID: 101 | Avatar Endgame       | Pukul: 14:00
ID: 102 | KKN di Desa Penari   | Pukul: 16:00
ID: 105 | Spiderman: LastGame  | Pukul: 17:00
ID: 108 | Home Alone           | Pukul: 20:00
=============================================
```

Soal 2	
1	

```
=== Kopi Wesker ===
1. Tambah Antrean Biasa (belakang)
2. Tambah Antrean VIP (depan)
3. Sisipkan Sebelum Nomor tertentu
4. Sisipkan Sesudah Nomor tertentu
5. Hapus Antrean Spesifik
6. Update Data Pesanan
0. Keluar
Pilihan Anda: 1
Masukkan Nomor Pelanggan: 1
Masukkan Nama Pelanggan: Novan
Masukkan Pesanan: Caffe Latte
DAFTAR PELANGGAN
=========================================
[1] Novan - Pesanan: Caffe Latte
=========================================
```

```
=== Kopi Wesker ===
1. Tambah Antrean Biasa (belakang)
2. Tambah Antrean VIP (depan)
3. Sisipkan Sebelum Nomor tertentu
4. Sisipkan Sesudah Nomor tertentu
5. Hapus Antrean Spesifik
6. Update Data Pesanan
0. Keluar
Pilihan Anda: 2
Masukkan Nomor Pelanggan: 2
Masukkan Nama Pelanggan: Mike
Masukkan Pesanan: Americano
Pelanggan berhasil ditambahkan di awal daftar!
DAFTAR PELANGGAN
=========================================
[2] Mike - Pesanan: Americano
[1] Novan - Pesanan: Caffe Latte
=========================================
```

```
Pilihan Anda: 3
Cari Nomor antrian Target :1
Masukkan Nomor Pelanggan: 3
Masukkan Nama Pelanggan: Aufaa
Masukkan Pesanan: Special Hot Chocolate
Pelanggan berhasil disisipkan sebelum Nomor 1!
DAFTAR PELANGGAN
==========================================
[2] Mike - Pesanan: Americano
[3] Aufaa - Pesanan: Special Hot Chocolate
[1] Novan - Pesanan: Caffe Latte
==========================================
```

```
Pilihan Anda: 4
Cari Nomor antrian Target :3
Masukkan Nomor Pelanggan: 4
Masukkan Nama Pelanggan: Zahid
Masukkan Pesanan: Matcha Latte
Pelanggan berhasil disisipkan setelah Nomor 3!
DAFTAR PELANGGAN
==========================================
[2] Mike - Pesanan: Americano
[3] Aufaa - Pesanan: Special Hot Chocolate
[4] Zahid - Pesanan: Matcha Latte
[1] Novan - Pesanan: Caffe Latte
==========================================
```

```
Pilihan Anda: 5
Cari Nomor antrian Target :1
Pelanggan dengan Nomor 1 berhasil dihapus!
DAFTAR PELANGGAN
==========================================
[2] Mike - Pesanan: Americano
[3] Aufaa - Pesanan: Special Hot Chocolate
[4] Zahid - Pesanan: Matcha Latte
==========================================
```

```
Pilihan Anda: 6
Cari Nomor antrian Target :2
Masukkan Pesanan Baru: Hazelnut Latte
Data pelanggan dengan Nomor 2 berhasil diperbarui!
DAFTAR PELANGGAN
==========================================
[2] Mike - Pesanan: Hazelnut Latte
[3] Aufaa - Pesanan: Special Hot Chocolate
[4] Zahid - Pesanan: Matcha Latte
==========================================
```